

0.76.2. Un déclencheur en (0, 0) : le B tourne en carré

Le 0.76.1, plus un bloc : une fois apparu, le B fait un carré sans fin à partir du milieu, un pas tous les quarts de seconde, sans bloquer le A.

1. Ce qu'il faut comprendre

C'est le 0.76.1, plus un bloc : les mouvements, qui ne commencent que quand `actif` vaut 1.

Pourquoi pas `carre` : `carre` bloque. Pendant son tour, le A ne pourrait plus bouger. On fait donc le carré **pas à pas**, avec `un_pas`, qui n'attend pas (le 0.30).

Le tour, compté par `bpas` : un carré de 4 cases de côté, c'est 16 pas. `bpas` va de 0 à 15 : de 0 à 3, un pas à droite ; de 4 à 7, en bas ; de 8 à 11, à gauche ; de 12 à 15, en haut. Après le pas 15, `bpas` revient à 0 : le tour recommence, sans fin.

`else if` enchaîne les cas : on ne teste le suivant que si le précédent était faux. Un seul des quatre `un_pas` est fait à chaque pas.

Le rythme : `bimages` compte les images ; à 15 (un quart de seconde), un pas, et on recompte. Le B part du milieu (10, 8) et tourne : (14, 8), (14, 12), (10, 12), puis retour au milieu.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_croix>` → [0.66.3. La fonction `deplace\_croix\(\)` — suivre la croix, case par case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <nombre>` → [0.8.3. La fonction `nombre\(\)` — écrire un nombre](#)
- `#include <poser>` → [0.1.3. La fonction `poser\(\)` — poser une tuile](#)
- `#include <un_pas>` → [0.29.4. La fonction `un\_pas\(\)` — un seul pas, sans attendre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1574 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	89 o	← la console, à l'allumage — la mise en route de la console
deplace_croix()	217 o	← main() — une fonction fournie par le langage
un_pas()	117 o	← main() — une fonction fournie par le langage
main()	259 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireNombre — attend de pouvoir écrire une case
AdresseCase	10 o	← effacer(), poser() — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() — efface des cases

EcrireNombre	33 o	← nombre()	– écrit un nombre en chiffres
CopierTuiles	18 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
LireManette	37 o	← bouton()	– lit les huit boutons
Tuiles	704 o	← CopierTuiles	– la police entière

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 3	32 o sur place, dans deplace_croix(), un_pas(), main() + AdresseCase (10 o)
effacer()	× 2	30 o sur place, dans deplace_croix(), un_pas() + AdresseCase (10 o), EffacerCases (12 o)
bouton()	× 4	20 o sur place, dans deplace_croix() + LireManette (37 o)
nombre()	× 2	20 o sur place, dans main() + EcrireNombre (33 o)
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)
images()	× 1	2 o sur place, dans deplace_croix()

Tes fonctions appelées :

un_pas()	× 4	88 o sur place, dans main()
deplace_croix()	× 1	19 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_croix> // une tuile qui suit la croix, case par case
#include <ALPHABET>      // les lettres de la police : ALPHABET[0] est le A
#include <nombre>         // écrit un nombre en chiffres
#include <poser>          // pose une tuile sur une case du fond
#include <un_pas>         // fait faire un seul pas à une tuile
```

2. UNE VARIABLE — X

```
uint8_t x = 10; // le A part de (10, 0)
```

3. UNE VARIABLE — Y

```
uint8_t y = 0;
```

4. UNE VARIABLE — ACTIF

```
uint8_t actif = 0; // 0 : pas encore ; 1 : le A est passé par (0, 0)
```

5. UNE VARIABLE — BX

```
uint8_t bx = 10; // le B : sa colonne...
```

6. UNE VARIABLE — BY

```
uint8_t by = 8; // ...et sa ligne. Il apparaît au milieu, (10, 8).
```

7. UNE VARIABLE — BPAS

```
uint8_t bpas = 0; // où il en est dans son tour : du pas 0 au pas 15
```

8. UNE VARIABLE — BIMAGES

```
uint8_t bimages = 0; // le chronomètre du B, en images
```

9. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

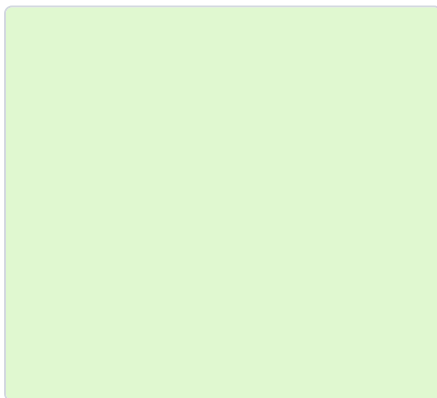
```
int main() {
  while (true) {
    image();
    deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
    nombre(0, 17, x);                      // sa position, en direct
    nombre(4, 17, y);

    // Le déclencheur : le A arrive en (0, 0), et actif vaut encore 0.
    if (x == 0 && y == 0 && actif == 0) {
      actif = 1;                          // une seule fois : ensuite actif vaut 1
      poser(10, 8, ALPHABET[1]);          // le B apparaît au milieu
    }

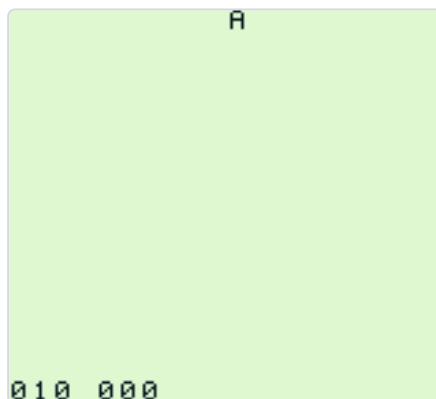
    // Les mouvements, une fois le déclencheur passé.
    if (actif == 1) {
      bimages++;                          // le chronomètre avance
      if (bimages == 15) {                 // toutes les 15 images : un pas
        bimages = 0;
        if (bpas < 4) { un_pas(bx, by, ALPHABET[1], 1, 0); } // pas 0 à 3 : à droite
        else if (bpas < 8) { un_pas(bx, by, ALPHABET[1], 0, 1); } // pas 4 à 7 : en bas
        else if (bpas < 12) { un_pas(bx, by, ALPHABET[1], -1, 0); } // pas 8 à 11 : à gauche
        else { un_pas(bx, by, ALPHABET[1], 0, -1); } // pas 12 à 15 : en haut
        bpas++;
        if (bpas == 16) { bpas = 0; } // le tour est fini : on recommence
      }
    }
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

En (0, 0), le B apparaît au milieu, puis tourne en carré sans fin, pendant que le A suit toujours la croix.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le B tourne : (14, 8), (14, 12), (10, 12)

✓ et revient au milieu, (10, 8), après le tour

Ensuite : **0.76.3. Un déclencheur en (0, 0) : le carré plus rapide** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche

Game Boy en C++